

AI Assisted Coding

Assignment 5.5

Name: Nihaal Krishna

Hallticket:2303A52498

Task-01:

Use AI to generate two solutions for checking prime numbers:

- Naive approach(basic)
- Optimized approach

Prompt:

“””Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”””

```
# Method 1: Basic Prime Checking
def is_prime_basic(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

# Method 2: Optimized Prime Checking
def is_prime_optimized(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False

    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6

    return True

# Example usage
number = int(input("Enter a number to check if it's prime: "))

if is_prime_basic(number):
    print(f"{number} is a prime number (Basic Check).")
else:
    print(f"{number} is not a prime number (Basic Check).")

if is_prime_optimized(number):
    print(f"{number} is a prime number (Optimized Check).")
else:
    print(f"{number} is not a prime number (Optimized Check).")

"""
Explanation:
The basic prime-checking method (is_prime_basic) checks for factors from 2 up to n-1,
which can be inefficient for large numbers.

The optimized method improves performance by:
1. Eliminating even numbers and multiples of 3 right away.
2. Checking factors only up to the square root of n.
3. Skipping even numbers in the loop by checking i and i + 2.

This significantly reduces the number of iterations, especially for larger numbers.
"""
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Enter a number to check if it's prime: 24
24 is not a prime number (Basic Check).
24 is not a prime number (Optimized Check).
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Enter a number to check if it's prime: 19
19 is a prime number (Basic Check).
19 is a prime number (Optimized Check).
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Enter a number to check if it's prime: 43
43 is a prime number (Basic Check).
43 is a prime number (Optimized Check).
PS C:\Users\nihaa> |
```

Task-02:

Prompt:

Here is a slightly modified, student-written version of your prompt:

Create a recursive Python function to calculate Fibonacci numbers. The code should be written clearly and correctly, with detailed comments explaining what recursion means and how the function calls itself. Clearly describe the base cases and the recursive case within the explanation. After writing the code, provide a complete step-by-step explanation of how the function executes when $n = 5$, strictly following the actual flow of the code without skipping any steps or giving vague theoretical explanations.

CODE:

```
# Recursive function to calculate Fibonacci numbers
def fibonacci(n):

    # Recursion is a technique where a function calls itself
    # to solve smaller parts of the same problem.

    # Base cases:
    # If n is 0, return 0
    if n == 0:
        return 0

    # If n is 1, return 1
    elif n == 1:
        return 1

    # Recursive case:
    # The function calls itself with (n-1) and (n-2)
    # and adds those results.
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

# Ask user for input
n = int(input("Enter a positive integer to find the nth Fibonacci number: "))

# Calculate Fibonacci
result = fibonacci(n)

# Display result
print(f"The {n}th Fibonacci number is: {result}")
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Enter a positive integer to find the nth Fibonacci number: 27
The 27th Fibonacci number is: 196418
PS C:\Users\nihaa> █
```

Justification

The provided code demonstrates structured and clear recursive problem-solving. It explicitly defines base cases to prevent infinite recursion and correctly handles the recursive case by breaking the problem into smaller subproblems. The comments explain how the function calls itself and how the solution builds up from smaller Fibonacci values. This clear separation of base and recursive logic improves readability, correctness, and understanding of recursion flow, making the program reliable and easy to maintain.

Task-03:

Use AI to generate a Python program that reads a file and processes data. Prompt:

“Generate code with proper error handling and clear explanations for each exception.”

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

Prompt:

Generate a Python program that reads a file and processes its contents using proper error handling mechanisms such as try, except, and finally. The code should include clear comments explaining each step, and detailed comments describing why each specific exception might occur and under what runtime conditions it would be triggered. After the code, provide a simple explanation of each exception scenario and ensure that the explanation accurately reflects the actual runtime behavior of the program.

Code:

```
# Generate code with proper error handling and clear explanations for each exception.
# and give Validation that explanations align with runtime behavior clearly in a tabular form.

def divide_numbers(a, b):
    """Divide two numbers with error handling."""
    try:
        result = a / b

    except ZeroDivisionError:
        return "Error: Cannot divide by zero."

    except NameError:
        return "Error: One of the variables is not defined."

    except ValueError:
        return "Error: Invalid value provided."

    except TypeError:
        return "Error: Both inputs must be numbers."

    except Exception as e:
        return f"An unexpected error occurred: {e}"

    else:
        return result

# Example usage
print(divide_numbers(10, 2))    # Valid case
print(divide_numbers(10, 0))    # Division by zero
print(divide_numbers(10, 'a'))  # Invalid type

# Value error example (not directly applicable in this context, but included for completeness)
print(divide_numbers(10, -5))   # This will not raise ValueError but included for demonstration

# Name error example
print(divide_numbers(10, None)) # This will not raise NameError but included for demonstration

# Explanation of error handling and validation
explanation_errors = """
| Exception Type | Explanation | Runtime Behavior Example |
|-----|-----|-----|
| ZeroDivisionError | Raised when attempting to divide by zero. | divide_numbers(10, 0) returns "Error: Cannot divide by zero." |
| TypeError | Raised when inputs are of incorrect type. | divide_numbers(10, 'a') returns "Error: Both inputs must be numbers." |
| NameError | Raised when a variable is not defined. | divide_numbers(10, None) returns "Error: One of the variables is not defined." |
| ValueError | Raised when an invalid value is passed. | divide_numbers(10, -5) returns "Error: Invalid value provided." |
| General Exception | Catches any other unexpected errors. | If an unexpected error occurs, it returns a message with error details. |
"""
print(explanation_errors)
```

OUTPUT:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/nihaa/OneDrive/Desktop/3rd yr 2nd sem/AI Assistant coding/assign 5.5/AI_5.5"
5.0
Error: Cannot divide by zero.
Error: Both inputs must be numbers.
-2.0
Error: Both inputs must be numbers.

| Exception Type | Explanation | Runtime Behavior Example |
|-----|-----|-----|
| ZeroDivisionError | Raised when attempting to divide by zero. | divide_numbers(10, 0) returns "Error: Cannot divide by zero." |
| TypeError | Raised when inputs are of incorrect type. | divide_numbers(10, 'a') returns "Error: Both inputs must be numbers." |
| NameError | Raised when a variable is not defined. | divide_numbers(10, None) returns "Error: One of the variables is not defined." |
| ValueError | Raised when an invalid value is passed. | divide_numbers(10, -5) returns "Error: Invalid value provided." |
| General Exception | Catches any other unexpected errors. | If an unexpected error occurs, it returns a message with error details. |

PS C:\Users\nihaa> 
```

Justification :

The provided code demonstrates robust error handling by catching specific exceptions that may arise during division operations. Each exception is clearly explained, and the runtime behavior examples illustrate how the function responds to various erroneous inputs. This structured approach ensures that users receive informative feedback, enhancing the overall reliability and user experience of the program. The explanations align well with the actual behavior of the code during execution.

Task_04:

Task: Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling practices.

Expected Output:

- Identification of security flaws (plain-text passwords, weak validation).
- Revised version using password hashing and input validation.

Prompt:

Generate a simple Python-based login system using an AI tool. The program should implement basic login verification logic without using any external authentication services, and the code should remain clear and easy to understand. After generating the code, carefully analyze whether the password handling method is secure. Identify and clearly explain any security weaknesses found in the implementation, such as storing passwords in plain text, absence of hashing or salting, weak input validation, or vulnerability to brute-force attacks. The explanation must directly reference specific parts of the generated code and explain why those practices are insecure in real-world systems, without relying on generic cybersecurity theory.

CODE:

```
import hashlib
import getpass

# Simple login system with secure password handling
users_db = {
    "user1": hashlib.sha256("password123".encode()).hexdigest(),
    "user2": hashlib.sha256("mysecurepassword".encode()).hexdigest()
}

def hash_password(password):
    """Hash a password using SHA-256."""
    return hashlib.sha256(password.encode()).hexdigest()

def verify_login(username, password):
    """Verify user login credentials."""
    if username in users_db:
        hashed_input_password = hash_password(password)
        if users_db[username] == hashed_input_password:
            return True
    return False

# Example usage
username = input("Enter username: ")
password = getpass.getpass("Enter password: ")

if verify_login(username, password):
    print("Login successful!")
else:
    print("Invalid username or password.")

# Analysis of Password Handling Practices:

# 1. Plain-text Password Storage:
# - The generated code does not store passwords in plain text; instead, it uses SHA-256 hashing.
# - This is a good practice as it prevents exposure of actual passwords if the database is compromised.

# 2. Lack of Hashing or Salting:
# - While the code uses hashing, it does not implement salting.
# - Salting adds random data to passwords before hashing, making it more secure against rainbow table attacks.
# - The absence of salting is a security flaw.

# 3. Weak Input Validation:
# - The code does not validate the username and password inputs for length, complexity, or format.
# - Weak input validation can lead to vulnerabilities such as injection attacks or acceptance of weak passwords.

# 4. Vulnerability to Brute-Force Attempts:
# - The code does not implement any mechanisms to prevent brute-force attacks, such as account lockout after multiple failed attempts or rate limiting.
# - This makes the system vulnerable to attackers trying multiple password combinations.

# In summary, while the generated code demonstrates some secure practices like hashing passwords,
# it lacks critical security measures such as salting, input validation, and protection against brute-force attacks,
# which are essential for a robust authentication system in real-world applications.
```

Output:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe  
Enter username: levi  
Enter password:  
Invalid username or password.
```

Justification:

The output shown in the terminal is correct according to the logic written in the program. The login system first checks whether the entered username exists inside the `users_db` dictionary. In this case, the username "levi" is not stored in the system, so the condition fails. Because of this, the function returns False and the program displays the message "Invalid username or password."

This shows that the login verification logic is working properly. The system does not specify whether the username or the password is wrong, which is actually a good security practice because it avoids giving extra information to potential attackers. The repeated password prompt appears because the program was executed again, not because of any loop in the code.

Task-05:

Task: Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

- Identified privacy risks in logging.
- Improved version with minimal, anonymized, or masked logging.
- Explanation of privacy-aware logging principles.

Prompt:

Create a Python program that records basic user login activity, including the username, IP address, and the date and time of login. After generating the program, analyze what kind of personal or sensitive information is being stored and explain why logging this type of data may create privacy or security risks. Then update the program to improve privacy by reducing unnecessary data collection, masking or partially anonymizing the IP address, and applying simple privacy-friendly practices in the logging process.

Code:

```
import logging
from datetime import datetime

# Configure logging
logging.basicConfig(
    filename='user_activity.log',
    level=logging.INFO,
    format='%(asctime)s - %(message)s'
)

def log_user_activity(username, ip_address):
    """Log user activity with masked IP address."""

    # Mask the IP address for privacy (e.g., replace last octet with 'xxx')
    masked_ip = '.'.join(ip_address.split('.')[:-1] + ['xxx'])

    # Log the username and masked IP address
    logging.info(f'User: {username}, IP: {masked_ip}')

# Example usage
username = input("Enter username: ")
ip_address = input("Enter IP address: ")
log_user_activity(username, ip_address)

# Analysis of Logged Data:

# 1. Personal or Sensitive Data Being Logged:
# - The program logs the username and IP address of the user.

# 2. Privacy or Security Risks:
# - Logging usernames can lead to privacy concerns if the log files are accessed by unauthorized individuals.
# - IP addresses can be used to track user locations and activities, posing security risks if exposed.

# Modifications Made:

# 1. Reduced Unnecessary Data Logging:
# - The program now only logs the username and a masked version of the IP address, omitting the full IP.

# 2. Masked or Anonymized IP Address:
# - The last octet of the IP address is replaced with 'xxx' to anonymize it, reducing the risk of tracking.

# 3. Followed Basic Privacy Rules:
# - Sensitive data exposure is minimized.
# - Only essential information is logged.
# - Partial anonymization helps balance monitoring and privacy.
```

OUTPUT:

```
PS C:\Users\nihaa> & C:/Users/nihaa/AppData/Local/Programs/Python/Python313/python.exe
Enter username: Nihaal
Enter username: Nihaal
Enter IP address: 0.0.0.0
```

Justification:

The modifications enhance user privacy by limiting the amount of sensitive information logged. Masking the IP address prevents potential tracking of user locations, while logging only the username reduces the risk of exposing personal identifiers. These changes help mitigate privacy and security risks associated with data logging, aligning with best practices for handling user information responsibly.

